

RAPPORT DE MISSION DEVOPS

Mise en place d'un environnement de developpement conteneurise

Auteur : [Votre Nom] **Formateur / Responsable :** [Nom du formateur]
Date : 08 octobre 2026 **Version :** 2.0

1. CONTEXTE ET OBJECTIFS

1.1 Contexte

Dans le cadre de ma formation DevOps / SysOps, j'ai ete charge de mettre en place un environnement de developpement local professionnel, base sur des technologies utilisees en entreprise : WSL2, Docker, PostgreSQL, et une API REST.

1.2 Objectifs

- Installer et configurer WSL2 avec la distribution Debian
 - Installer et configurer Docker Engine avec un utilisateur dedie
 - Dockeriser une API FastAPI existante (portfolio-auth-api)
 - Mettre en place une base de donnees PostgreSQL conteneurisee
 - Valider le flux d'authentification JWT
 - Preparer un environnement de production
 - Deployer l'API sur Internet de maniere gratuite et securisee
-

2. ENVIRONNEMENT TECHNIQUE

Composant	Version	Role
Windows	11	Systeme hote
WSL2	2	Sous-systeme Linux
Debian	13 (trixie)	Distribution Linux
Docker Engine	29.7.2	Conteneurisation
Python	3.13	Langage de l'API
FastAPI	0.141.1	Framework API REST
PostgreSQL	15-alpine	Base de donnees
Adminer	latest	Interface de gestion DB
Node.js	20.19.2	Environnement JS
localtunnel	2.0.2	Tunnel public

3. TRAVAUX REALISES

3.1 Phase 1 : Installation et configuration de WSL2

Actions menees : - Verification des distributions WSL disponibles - Choix de Debian comme environnement de travail - Gestion de l'utilisateur `admin` (reinitialisation du mot de passe via `root`) - Mise a jour des depots APT

Commandes cles : `wsl --list --verbose wsl --d Debian wsl --d Debian --u root`
`passwd admin sudo apt update`

Resultat : Environnement Linux fonctionnel avec utilisateur `admin`.

3.2 Phase 2 : Installation de Docker Engine

Actions menees : - Ajout du depot officiel Docker (cle GPG + sources) - Installation de `docker-ce`, `docker-ce-cli`, `containerd.io` - Ajout de l'utilisateur `admin` au groupe `docker` - Test avec `hello-world`

Commandes cles : `sudo apt install docker-ce docker-ce-cli containerd.io sudo`
`usermod -aG docker $USER docker run hello-world`

Resultat : Docker operationnel, utilisateur `admin` autorise sans `sudo`.

3.3 Phase 3 : Dockerisation de l'API FastAPI

Actions menees : - Creation du Dockerfile (Python 3.13-slim) - Creation du `docker-compose.yml` (3 services : `db`, `api`, `adminer`) - Configuration du reseau Docker dedie (`app-network`) - Configuration des volumes pour la persistance

Problemes resolus : - Erreur `No module named 'app'` : ajout de `--app-dir /app` dans le CMD - Erreur de resolution DNS (`db` resolu en `192.168.1.1`) : creation d'un reseau dedie

Resultat : API accessible sur `localhost:8000`, base sur `5433`, Adminer sur `8080`.

3.4 Phase 4 : Sauvegarde et Restauration (Disaster Recovery)

Actions menees : - Sauvegarde de la base avec `pg_dump` - Simulation d'un crash (suppression de la base) - Gestion des connexions actives (`pg_terminate_backend`) - Restauration complete de la base

Commandes cles : `docker exec -it portfolio-db pg_dump -U postgres -d portfolio_db > backup.sql` `docker exec -it portfolio-db psql -U postgres -c "DROP DATABASE portfolio_db;"` `docker exec -i portfolio-db psql -U postgres -d portfolio_db < backup.sql`

Resultat : Procedure de sauvegarde/restauration maitrisee et testee.

3.5 Option A : Validation du flux JWT

Actions menees : - Creation d'un utilisateur via /auth/register - Recuperation d'un token JWT via /auth/login - Test de la route protegee /auth/me avec curl

Resultat : Flux d'authentification complet valide.

3.6 Option B : Exploration avec Adminer

Actions menees : - Connexion a Adminer via l'IP interne (172.19.0.4) - Visualisation des tables users et products - Verification de la persistance des donnees

Resultat : Preuve visuelle des donnees en base.

3.7 Option C : Extension de l'API (Module Products)

Actions menees : - Ajout du modele SQLAlchemy Product - Ajout des schemas Pydantic (ProductCreate, ProductResponse) - Creation des routes CRUD (POST /products, GET /products) - Enregistrement dans main.py

Problemes resolus : - Erreur `NameError: Float is not defined` : import corrige - Erreur `422 JSON decode error` : envoi de deux objets JSON

Resultat : API extensible avec un nouveau module metier.

3.8 Option D : Preparation a la production

Actions menees : - Creation de docker-compose.prod.yml - Suppression du mode `-reload` - Suppression des volumes de code - Separation claire Dev / Prod

Resultat : Configuration de production prete pour un VPS.

3.9 Option E : Documentation

Actions menees : - Mise a jour du fichier `commande_linux_devops_p001.md` - Generation du PDF correspondant via pandoc

Resultat : Livrable documentaire complet.

3.10 Deploiement public avec localtunnel

Actions menees : - Installation de Node.js 20 et npm - Installation de localtunnel - Creation du tunnel vers le port 8000 - Obtention d'une URL publique stable

URL publique : <https://portfolio-herslinux.loca.lt/docs>

Resultat : API accessible depuis n'importe ou dans le monde, gratuitement.

3.11 Option F : Sécurisation de l'API

Actions menees : - Ajout d'un middleware Basic Auth - Creation des identifiants dans .env - Exclusion des routes de documentation (/docs) - Protection des routes metier (/auth/me, /products)

Problemes resolus : - Erreur `NameError: BaseHTTPMiddleware` : fichier reecrit proprement - Fichier .env mal formate : separation des lignes

Resultat : API securisee, documentation accessible, routes protegees.

4. DIFFICULTES RENCONTREES ET SOLUTIONS

#	Probleme	Cause	Solution
1	<code>lsb_release: command not found</code>	Paquet non installe	Utilisation directe de <code>trixie</code>
2	<code>No module named 'app'</code>	Chemin Python incorrect	Ajout de <code>--app-dir /app</code>
3	<code>db resolu en 192.168.1.1</code>	DNS externe prioritaire	Reseau Docker dedie
4	<code>address already in use (port 5432)</code>	Conflit de port	Changement vers 5433
5	<code>database is being accessed</code>	Connexions actives	<code>pg_terminate_backend</code>
6	<code>Float is not defined</code>	Import manquant	Ajout de Float dans <code>sqlalchemy</code>
7	<code>JSON decode error</code>	Deux objets JSON envoyes	Un seul objet par requete
8	<code>SECRET_KEY</code> avec <code>\$</code>	Interpretation shell	Echappement / guillemets
9	<code>BaseHTTPMiddleware</code> non defini	Imports mal places	Reecriture propre du fichier
10	<code>.env</code> mal formate	Lignes collees	Separation des variables

5. COMPETENCES ACQUISES

5.1 Competences techniques

- Administration Linux (utilisateurs, paquets, services)
- Conteneurisation avec Docker et Docker Compose
- Gestion des reseaux Docker et resolution DNS
- Sauvegarde et restauration PostgreSQL
- Authentification JWT et Basic Auth
- Configuration multi-environnements (Dev / Prod)
- Deploiement public via tunnel
- Resolution de problemes complexes (debugging)

5.2 Competences transverses

- Methode de travail structuree (planification, validation, execution)
- Documentation technique
- Reflexe de securite ("Security by Design")
- Perseverance face aux erreurs

6. ETAT ACTUEL DE L'ENVIRONNEMENT

Element	Statut	Acces
API FastAPI (dev)	Operationnelle	localhost:8000
API FastAPI (public)	Operationnelle	https://portfolio-herslinux.localt
PostgreSQL	Operationnelle	localhost:5433
Adminer	Operationnel	localhost:8080
Middleware Basic Auth	Actif	Identifiants dans .env
Documentation Swagger	Accessible	/docs (sans mot de passe)
Routes metier	Protegees	Necessitent identifiants
Sauvegarde	Disponible	backup.sql

7. PROCHAINES ETAPES PREVUES

Option	Titre	Objectif
G	Reverse Proxy Nginx	Router plusieurs services via un point unique

Option	Titre	Objectif
H	CI/CD GitHub Actions	Automatiser tests et deploiements
I	Monitoring Prometheus + Grafana	Surveiller l'API et detecter les anomalies
J	MLOps (IA + DevOps)	Integrer un modele de Machine Learning

8. CONCLUSION

Cette mission m'a permis de mettre en pratique l'ensemble des competences fondamentales du DevOps : de l'installation d'un environnement Linux a la securisation d'une API publique, en passant par la conteneurisation, la sauvegarde et la documentation.

L'approche methodique (planification, execution, verification, validation) a ete essentielle pour surmonter les nombreuses difficultes rencontrees. Chaque bug resolu a renforce ma comprehension des technologies utilisees.

L'environnement produit est stable, securise et extensible. Il constitue une base solide pour les projets suivants (reverse proxy, CI/CD, monitoring, IA).

Statut de la mission : Objectifs atteints et valides.

Rapport redige dans le cadre de la formation DevOps / SysOps. Toutes les commandes utilisees sont documentees dans le fichier annexe : commande_linux_devops_p001.pdf

FIN DU RAPPORT